



NATIONAL SENIOR CERTIFICATE EXAMINATION
NOVEMBER 2010

INFORMATION TECHNOLOGY: PAPER II
MARKING GUIDELINES

Time: 3 hours

120 marks

These marking guidelines were used as the basis for the official IEB marking session. They were prepared for use by examiners and sub-examiners, all of whom were required to attend a rigorous standardisation meeting to ensure that the guidelines were consistently and fairly interpreted and applied in the marking of candidates' scripts.

At standardisation meetings, decisions are taken regarding the allocation of marks in the interests of fairness to all candidates in the context of an entirely summative assessment.

The IEB will not enter into any discussions or correspondence about any marking guidelines. It is acknowledged that there may be different views about some matters of emphasis or detail in the guidelines, and different interpretations of the application thereof. Hence, the specific mark allocations have been omitted.

Model Answer: Question 1

1.

```
SELECT * FROM tblCars WHERE CarYear = 2010
```

2.

```
SELECT * FROM tblCars WHERE CarRegistration LIKE '%GP' (any  
working method)  
OR  
SELECT * FROM tblCars WHERE RIGHT (CarRegistration, 2) = 'GP'
```

3.

```
SELECT COUNT(*) AS qryCount FROM tblCars WHERE CarModel =  
'Volkswagen Polo';
```

4.

```
SELECT tblCars.* FROM tblCars INNER JOIN tblOwners ON  
tblCars.OwnerID = tblOwners.OwnerID WHERE OwnerAge >= 45;
```

5.

```
SELECT SUM(FineAmount) AS qryTotal FROM tblFines INNER JOIN  
tblCars ON tblCars.CarRegistration = tblFines.CarRegistration  
WHERE CarYear > 2008;
```

6.

```
SELECT SUM(FineAmount) AS qryTotal FROM tblFines GROUP BY  
CarRegistration;
```

7.

```
UPDATE tblFines SET FineAmount = FineAmount / 2 WHERE NOW() -  
FineDate < 32; -- Access (overall syntax)
```

```
UPDATE tblFines SET FineAmount = FineAmount / 2 WHERE NOW() -  
FineDate < 32 * 86400 -- MySQL (overall syntax)
```

8.

```
DELETE FROM tblFines WHERE FineOfficerID = 'TPDW40NN5';  
(overall syntax)
```

Model Answer: Question 2

```
/**
 * Model Answer
 * Date: 7 February 2010
 * Questions: 2.1, 2.2, 2.3, 2.4, 2.5, 2.6
 * @author philip
 */

// Question 2.1
public class NationalLicence
{
    // Question 2.2

    private int year;
    private String licence;
    // End 2.2

    // Question 2.3
    public NationalLicence (int y, String l)
    {
        year = y;
        licence = l;
    }
    // End 2.3

    // Question 2.4
    public int getYear ()
    {
        return year;
    }

    public String getLicence ()
    {
        return licence;
    }
    // End 2.4

    // Question 2.5
    public int getLicenceFee ()
    {
        return 250;
    }
    // End 2.5

    // Question 2.6
    public String toString ()
    {
        return licence + "\t" + getLicenceFee ();
    }
    // End 2.6
}
```

```
/**
 * Model Answer
 * Date: 7 February 2010
 * Questions: 4.1, 4.2, 4.3
 * @author philip
 */

// Question 4.1
public class ZNLicence extends NationalLicence
{
    // Question 4.2
    public ZNLicence (int y, String l)
    {
        super (y, l);
    }
    // End 4.2

    // Question 4.3
    public int getLicenceFee ()
    {
        if (getYear () < 2000)
        {
            return super.getLicenceFee () + 50;
        }
        else
        {
            return super.getLicenceFee ();
        }
    }
    // End 4.3
}
```

```
/**
 * Model Answer
 * Date: 7 February 2010
 * Questions: 3.1, 3.2, 7
 * @author philip
 */

// Question 3.1
public class GPLicence extends NationalLicence
{
    // Question 3.2
    public GPLicence (int y, String l)
    {
        super (y, l);
    }
    // End 3.2

    // Question 7
    public int getLicenceFee ()
    {
        double licenceFee = super.getLicenceFee () * 1.2;

        String vowels = "AEIOU";
        boolean personalised = false;

        for (int i = 0 ; i < 3; i ++ )
        {
            if (vowels.indexOf (getLicence ().charAt (i)) >= 0)
            {
                licenceFee *= 2;
                personalised = true;
            }
        }

        if (personalised)
        {
            int numbers = Integer.parseInt (getLicence
            ().substring (3, 6));
            licenceFee += numbers;
        }

        return (int) licenceFee;
    }
    // End 7
}
```

```
/**
 * Model Answer
 * Date: 7 February 2010
 * Questions: 5.1, 5.2, 5.3, 5.4, 5.5, 5.6
 * @author philip
 */
import java.io.*;

// Question 5.1
public class LicenceManager
{
    private NationalLicence[] licences = new NationalLicence[200];
    private int counter = 0;
    // End 5.1

    // Question 5.2
    public LicenceManager ()
    {
        try
        {
            BufferedReader fr = new BufferedReader (new FileReader
("licence.txt"));

            String line = fr.readLine ();

            while (line != null)
            {
                String[] details = line.split("#");
                int year = Integer.parseInt (details[0]);
                String licence = details[1];

                if (licence.substring (6, 8).equalsIgnoreCase ("GP"))
                {
                    licences[counter] = new GPLicence (year, licence);
                    counter++;
                }
                else
                {
                    licences[counter] = new ZNLicence (year, licence);
                    counter++;
                }

                line = fr.readLine ();
            }
        }
        catch (IOException e)
        {
            System.out.println (e);
        }
    }
    // End 5.2
}
```

```
// Question 5.3
public int totalLicenceFees ()
{
    int total = 0;
    for (int i = 0; i < counter; i++)
    {
        total += licences [i].getLicenceFee ();
    }

    return total;
}
// End 5.3

// Question 5.4
public String getGPLicenceList ()
{
    String gpCars = "";
    for (int i = 0; i < counter; i++)
    {
        if (licences [i] instanceof GPLicence)
        {
            gpCars = gpCars + licences [i].toString () + "\n";
        }
    }

    return gpCars;
}
// End 5.4

// Question 5.5
public String getHighestLicence ()
{
    int max = 0;

    for (int i = 1; i < counter; i ++)
    {
        if (licences [i].getLicenceFee () > licences
[max].getLicenceFee ())
        {
            max = i;
        }
    }

    return licences [max].toString ();
}
// End 5.5
}
```

```
/*  
  
 * To change this template, choose Tools | Templates  
 * and open the template in the editor.  
 */  
  
/**  
 *  
 * @author philip  
 */  
public class Interface  
{  
    public static void main (String[] args)  
    {  
        LicenceManager lm = new LicenceManager ();  
  
        System.out.println ("Total Licences:\n" + lm.totalLicenceFees  
());  
        System.out.println ("GP List:\n" + lm.getGPLicenceList ());  
        System.out.println ("Highest Licence:\n" + lm.getHighestLicence  
());  
    }  
}
```

Delphi Solution

```

unit uNationalLicense;

interface

uses SysUtils, Math, Dialogs;

type classNationalLicense = class (TObject)
    private
        fYear : Integer;
        fPlate : string;
    protected
    public
        constructor Create (y : Integer;
np : string);

        function GetYear : Integer;
        function GetNoPlate : string;
        function GetLicenseFee : Real;

virtual;

        function toString : string;
end;

implementation

{ classNationalLicense }

constructor classNationalLicense.Create(y: Integer; np: string);
begin
    fYear := y;
    fPlate := np;
end;

function classNationalLicense.GetLicenseFee: Real;
begin
    Result := 250;
end;

function classNationalLicense.GetNoPlate: string;
begin
    Result := fPlate;
end;

function classNationalLicense.GetYear: Integer;
begin
    Result := fYear;
end;

function classNationalLicense.toString: string;
begin
    Result := fPlate + #9 + FloatToStrF (GetLicenseFee, ffCurrency,
7, 2);
end;

end.

```

```

unit uGPLicense;

interface

uses uNationalLicense, SysUtils, Math, Dialogs;

type classGPLicense = class (classNationalLicense)
    private
    protected
    public
        constructor Create (y : Integer; np :
string);
        function GetLicenseFee : Real; override;
    end;

implementation

{ classGPLicense }

constructor classGPLicense.Create(y: Integer; np: string);
begin
    inherited Create(y, np);
end;

function classGPLicense.GetLicenseFee: Real;
var vowels : set of Char;
    index : Integer;
    LicenceFee : Real;
    VowelCount : Integer;
    CurPlate : string;
begin
    CurPlate := uppercase (GetNoPlate);
    if CurPlate = '' then
        begin
            Result := 0;
            Exit;
        end;
    VowelCount := 0;
    vowels := ['A', 'E', 'I', 'O', 'U'];
    LicenceFee := 250 * 1.2;
    for index := 1 to 3 do
        begin
            if CurPlate [index] in vowels then
                begin
                    inc (VowelCount);
                end;
            end;
        for index := 1 to VowelCount do
            begin
                LicenceFee := LicenceFee * 2;
            end;
        if VowelCount > 0 then
            begin
                LicenceFee := LicenceFee + StrToInt(copy(CurPlate ,4, 3));
            end;
        Result := LicenceFee;
    end;

end.

```

```
unit uZNLICENSE;  
  
interface  
  
uses uNationalLicense, SysUtils, Math, Dialogs;  
  
type classZNLICENSE = class (classNationalLicense)  
    private  
    protected  
    public  
        constructor Create (y : Integer; np :  
string);  
        function GetLicenseFee : Real; override;  
end;  
  
implementation  
  
{ classZNLICENSE }  
  
constructor classZNLICENSE.Create(y: Integer; np: string);  
begin  
    inherited Create(y, np);  
end;  
  
function classZNLICENSE.GetLicenseFee: Real;  
begin  
    if GetYear < 2000 then Result := inherited Get License Fee + 50  
        else Result := inherited Get License Fee  
end;  
  
end.
```

```

unit uLicenseManager;

interface

uses uGPLicense, uNationalLicense, uZNLICENSE, Math, SysUtils,
Dialogs;

type classLicenseManager = class (TObject)
    private
        fNoPlates : array [1..200] of
classNationalLicense;
        fItemCount : Integer;
    protected
    public
        constructor create;
        function TotalLicenseFees : real;
        function GetGPLicenseList : string;
        function HighestLicense : string;
    end;

implementation

{ classLicenseManager }

constructor classLicenseManager.create;
var tf : TextFile;
    tmp : string;
    yr : Integer;
begin
    fItemCount := 0;
    AssignFile(tf, 'licence.txt' );
    Reset(tf);
    while not Eof(tf) do
        begin
            inc (fItemCount);
            Readln(tf, tmp);
            yr := StrToInt(Copy (tmp, 1, 4));
            Delete (tmp, 1, 5);
            if UpperCase(tmp[8])= 'P' then
                begin
                    fNoPlates[fItemCount] := classGPLicense.Create(yr, tmp);
                end
            else
                begin
                    fNoPlates[fItemCount] := classZNLICENSE.Create(yr, tmp);
                end;
            end;
        end;
    CloseFile(tf);
end;

function classLicenseManager.GetGPLicenseList: string;
var GPList : string;
    index : Integer;
    TotFees : Real;

```

```
begin
  GPList := '';
  for index := 1 to fItemCount do
    begin
      if fNoPlates[index] is classGPLicense then
        begin
          TotFees := TotFees + fNoPlates[index].GetLicenseFee;
          GPList := GPList + fNoPlates[index].toString + #13;
        end;
      end;
    GPList := GPList + 'Total Due : ' + FloatToStrF(TotFees,
ffCurrency, 7, 2);
    Result := GPList;
  end;

function classLicenseManager.HighestLicense: string;
var index : Integer;
    HighFee : Real;
    HighPlate : string;
begin
  HighFee := 0;
  HighPlate := '';
  for index := 1 to fItemCount do
    begin
      if fNoPlates[index].GetLicenseFee > HighFee then
        begin
          HighFee := fNoPlates[index].GetLicenseFee;
          HighPlate := fNoPlates[index].toString;
        end;
      end;
    Result := 'Highest License Fee : ' + HighPlate;
  end;

function classLicenseManager.TotalLicenseFees: real;
var index : Integer;
    TotalFees : Real;
begin
  TotalFees := 0;
  for index := 1 to fItemCount do
    begin
      TotalFees := TotalFees + fNoPlates[index].GetLicenseFee;
    end;
  Result := TotalFees;
end;

end.
```

```
unit ufrmMain;

interface

uses
    Windows, Messages, SysUtils, Variants, Classes, Graphics,
    Controls, Forms,
    Dialogs, StdCtrls, ComCtrls;

type
    TForm1 = class(TForm)
        redt1: TRichEdit;
        procedure FormCreate(Sender: TObject);
    private
        { Private declarations }
    public
        { Public declarations }
    end;

var
    Form1: TForm1;

implementation

uses uLicenseManager;

var Licenses : classLicenseManager;

{$R *.dfm}

procedure TForm1.FormCreate(Sender: TObject);
begin
    Licenses := classLicenseManager.create;
    redt1.Lines.Clear;
    redt1.Lines.Add('Total License Fees : ' +
        FloatToStrF(Licenses.TotalLicenseFees,
    ffCurrency, 7, 2));
    redt1.Lines.Add('');
    redt1.Lines.Add('List of Gauteng Licenses:');
    redt1.Lines.Add('');
    redt1.Lines.Add(Licenses.GetGPLicenseList);
    redt1.Lines.Add('');
    redt1.Lines.Add(Licenses.HighestLicense);
end;

end.
```

Centre Number:	
Candidate Number:	
Marker's Name:	

TOTAL:**0****0%**

Q	Marking Guide	Mark	Pupil
1.1	SELECT * FROM tblCars WHERE CarYear = 2010; (overall syntax)	3	
1.2	SELECT * FROM tblCars WHERE CarRegistration LIKE '%GP' (any working method) OR: SELECT * FROM tblCars WHERE RIGHT (CarRegistration, 2) = 'GP'	5	
1.3	SELECT COUNT(*) AS qryCount FROM tblCars WHERE CarModel = 'Volkswagen Polo';	4	
1.4	SELECT tblCars.* FROM tblCars INNER JOIN tblOwners ON tblCars.OwnerID = tblOwners.OwnerID WHERE OwnerAge >= 45;	6	
1.5	SELECT SUM(FineAmount) AS qryTotal FROM tblFines INNER JOIN tblCars ON tblCars.CarRegistration = tblFines.CarRegistration WHERE CarYear > 2008;	7	
1.6	SELECT SUM(FineAmount) AS qryTotal FROM tblFines GROUP BY CarRegistration;	5	
1.7	<u>Access:</u> UPDATE tblFines SET FineAmount = FineAmount / 2 WHERE NOW() - FineDate < 32; (overall syntax)	6	
	<u>MySQL:</u> UPDATE tblFines SET FineAmount = FineAmount / 2 WHERE NOW() - FineDate < 32 * 86400 (overall syntax)		
1.8	DELETE FROM tblFines WHERE FineOfficerID = 'TPDW40NN5'; (overall syntax)	4	
2.1	Class name, correct file name (Java) / Unit (Delphi)	2	
2.2	Appropriate names, appropriate types, private	3	
2.3	Constructor method named, parameters, correct assignment of parameters	4	
2.4	Methods (both): named appropriately, correct return types, correct return statements	3	
2.5	Method typed appropriately (int or double), return 250	2	
2.6	Method typed correctly (String), return both properties , formatted with a tab.	4	
3.1	Correct class declaration, correct inheritance	2	
3.2	Constructor named correctly, correct parameters, calls parent constructor	3	
4.1	Correct class declaration, correct inheritance	2	
4.2	Constructor identical in structure to 3.2 - even if incorrect, works correctly	2	
4.3	Override done correctly (same name, return type & parameters) , implements an if-statement with the correct condition (year < 2000 or year >= 2000) return values based on parent class (super.getLicenceFee() or super.getLicenceFee()+50) (do not give marks for hard-coded values: 250 or 300)	5	

5.1	Class is named correctly, array declared, named appropriately (100 elements) typed as NationalLicence, integer counter variable	5	
5.2	Constructor created correctly, File opened for reading, loop to cycle through file with correct termination condition. Correct parsing of each line read correct determination of province (any working method), correct instantiation, counter increment	9	
5.3	Method typed appropriately, sum variable keep count correct for-loop to cycle through array, sum calculated correctly and returned	6	
5.4	For-loop, if-statement with instanceof concatenation to temp var formatted with newline value returned	6	
5.5	Appropriately named String typed for-loop check for higher remember higher return highest	6	
6	Interface created correctly calls each method in turn some (any!) output is visible correct output	6	
7	Method is overridden correctly Three criteria: Vowels are counted, integer is extracted, calculation is attempted with vowel count and integer. Non-personalised plates accounted for. Calculation is returned	10	